P9 PRESENTATION SCRIPT & DEFENSE GUIDE

บทพูดนำเสนอ P9 แบบเจาะลึกทีละหน้า (19 สไลด์)

Java OOP & Text Mode Simulator — ร้านข้าวต้มอารมณ์ดี (16 Model Classes)
คู่มือคำพูดคำต่อคำ, เทคนิคการชี้หน้าจอ (Visual Cues), ทฤษฎีปรับปรุงใหม่เข้าใจง่าย และคลังตอบคำถามอาจารย์

🎙️ คำพูดคำต่อคำ (Verbatim Script)
⏱️ ควบคุมเวลาเป๊ะ 6 – 8 นาที
👁️ จุดชี้หน้าจอ & Visual Cues
💎 พิสูจน์ Composition 5 คู่แท้
🛡️ คลังคำถามปราบเซียน (Q&A Defense)

วิชา: Object-Oriented Analysis and Design (OOAD)
ระบบ: SOP-based Restaurant Management System (ร้านข้าวต้มอารมณ์ดี)
โครงสร้าง: Main.java + 16 Model Classes (No Framework)

ฉบับสมบูรณ์: พฤศจิกายน 2026
สถานะ: พร้อมนำเสนอหน้าชั้นเรียน 100%

⏱️ แผนผังการคุมเวลาและโครงสร้างการนำเสนอ 3 องค์ (Presentation Roadmap)

การนำเสนอ P9 ถูกแบ่งออกเป็น 3 องค์หลักอย่างเป็นขั้นเป็นตอน เพื่อให้ผู้ฟังและอาจารย์เข้าใจได้ทันทีว่า *"ทฤษฎีคืออะไร -> โปรแกรมรันได้จริงอย่างไร -> และโค้ดพิสูจน์ความถูกต้องตามหลักการออกแบบอย่างไร"*

องค์ (Act)	หน้าสไลด์	เวลาแนะนำ	เป้าหมายและจุดเน้น
Act 1: ทฤษฎี OOP & Composition	หน้า 1 – 3 (สไลด์ 328–329)	1.5 – 2.0 นาที	ปูพื้นฐาน Composition vs Association, กฎเหล็ก 4 ข้อ, ยกตัวอย่างโค้ดจริงใน Order.java
Act 2: สาธิต 8 Use Cases (FR1–FR8)	หน้า 4 – 13 (สไลด์ 331–339)	3.0 – 3.5 นาที	แสดงความสมบูรณ์ของ Text Mode Simulator, Two-Pass Stock Check, KDS, SOP และระบบแต้ม
Act 3: พิสูจน์ Composition 5 คู่แท้	หน้า 14 – 19 (สไลด์ 340–344)	2.5 – 3.0 นาที	เจาะลึก 5 คู่แท้, การเรียก destroy() และ items.clear(), อธิบายว่าทำไมลูกตายตามแม่
ช่วงตอบคำถาม (Q&A Defense)	-	2.0 – 3.0 นาที	ใช้คลังคำตอบทางเทคนิคด้านหลังเอกสารเพื่อตอบอาจารย์อย่างมั่นใจ

💡 **4 กฎทองสำหรับผู้พูด (Golden Rules for Presenter):**
1. **อย่าอ่านสไลด์ตามตัวหนังสือ:** ให้ใช้สายตามองอาจารย์ แล้วชี้หน้าจอตามจุด `[ชี้ที่ ... ]` ที่ระบุในสคริปต์
2. **เน้นคำว่า "ลูกตายตามแม่":** ทุกครั้งที่พูดถึง Composition ให้ย้ำว่า *"เมื่อตัวแม่ถูกทำลาย ตัวลูกต้องหายไปด้วยทันที"*
3. **ย้ำว่า Table และ Customer เป็น Association:** เพื่อแสดงให้อาจารย์เห็นว่าเราแยกแยะความสัมพันธ์ 2 ชนิดนี้ออกจากกันอย่างแม่นยำ
4. **ชูจุดเด่น Clean OOP:** ย้ำว่าโปรเจกต์นี้เขียนด้วย Core Java ล้วนๆ ไม่มี Spring Boot หรือ Framework บังสายตา

ACT 1: ทฤษฎี OOP และการนำ Composition ไปใช้จริง หน้า 1 – 3 | รวมเวลา ~1.5 - 2 นาที

หน้า 1: หน้าปก P9

⏱️ เวลาพูด: ~15-20 วินาที • 🎯 วัตถุประสงค์: แนะนำตัว เปิดประเด็น และบอกขอบเขตผลงาน

สไลด์ที่ 1 | PDF หน้า 1

👁️ **ท่าทางและการชี้จอ:** ยืนตัวตรง สบตาอาจารย์ ผายมือไปที่หน้าจอ ไม่ต้องก้มอ่านกระดาษ

"กราบเรียนอาจารย์และสวัสดีเพื่อนๆ ทุกคนครับ วันนี้กลุ่มของพวกเราจะมานำเสนอ **Phase 9: Java OOP Implementation** ของระบบบริหารจัดการร้านข้าวต้มอารมณ์ดีครับ"

"ในเฟสนี้ เราได้นำผลการวิเคราะห์และออกแบบจาก Phase 1 ถึง Phase 8 มาแปลงเป็นโปรแกรมที่รันได้จริงบนภาษา Java โดยยึดหลัก **Pure OOP พื้นฐาน** ไม่มีการใช้ Framework หรือ Library เสริม เพื่อแสดงให้เห็นกลไกการทำงานของ Object, Data Encapsulation, และความสัมพันธ์ระหว่างคลาสอย่างแท้จริงครับ"

คีย์เวิร์ดสำคัญ: Pure OOP พื้นฐาน, ไม่ใช้ Framework, 16 Model Classes + Main.java

หน้า 2 (สไลด์ 328): ทฤษฎีการเขียน Code แบบ Composition

สไลด์ที่ 2 | สไลด์ 328 (Canva)

⏱️ เวลาพูด: ~45-50 วินาที • 🎯 วัตถุประสงค์: ปูทฤษฎีให้เข้าใจง่าย คมชัด ไม่สับสน

👁️ **ท่าทางและการชี้จอ:** 1. ชี้ที่เครื่องหมายข้าวหลามตัดทึบ (◆)
2. ชี้ที่การเปรียบเทียบ Composition vs Association
3. ชี้ที่ Checklist กฎเหล็ก 4 ข้อ

"ก่อนที่เราจะไปดูผลการรันโปรแกรม เราขอเริ่มที่หัวใจสำคัญทางทฤษฎีของระบบ นั่นคือ **Composition** ครับ"

"หลายคนอาจจะคุ้นชินกับนิยามยาวๆ แต่ถ้าสรุปให้เข้าใจง่ายที่สุดในประโยคเดียว: **Composition คือความสัมพันธ์แบบ 'ความเป็นเจ้าของร่วมตาย' ครับ**"

"หมายความว่า: **เมื่อใดก็ตามที่ไม่มีตัวแม่ ตัวลูกที่เป็นส่วนประกอบย่อยก็อยู่ไม่ได้ ต้องหายไปด้วยกันทันที** โดยสัญลักษณ์ข้าวหลามตัดทึบ (◆) จะวางอยู่ฝั่งตัวแม่เสมอ เช่น `Order ◆— OrderItem` "

"ถ้าเทียบให้เห็นภาพในร้านเราเลยครับ: ถ้าเรากดยกเลิกหรือฉีก **ใบออเดอร์ (Order)** ทิ้ง รายการอาหารในบิลนั้น **(OrderItem)** ก็ต้องหายตามไปด้วยทันที นี่คือ Composition แต่ในทางตรงกันข้าม **โต๊ะอาหาร (Table)** จะไม่หายตาม เพราะโต๊ะเป็นแค่ **Association (แค่อ้างถึง)**"

"ซึ่งในการเขียนโค้ดภาษา Java เรายึดกฎเหล็ก 4 ข้ออย่างเคร่งครัด ได้แก่: แม่เก็บลูกเป็น `private` , แม่สั่ง `new` สร้างลูกเอง, คลาสอื่นห้ามยุ่งกับลูกตรงๆ, และเมื่อแม่ถูกลบ ตัวแม่จะสั่งทำลายลูกทิ้งทั้งหมดครับ"

📝 ข้อความฉบับเข้าใจง่าย (คัดลอกไปอัปเดตสไลด์ 328 ใน Canva ได้เลย):

นิยาม: Composition คือ "ความเป็นเจ้าของร่วมตาย" (ถ้าไม่มีตัวแม่ ตัวลูกต้องหายไปด้วยกันทันที)
สัญลักษณ์: ข้าวหลามตัดทึบ (◆) อยู่ฝั่งตัวแม่เสมอ เช่น `Order ◆— OrderItem`
กฎโค้ด 4 ข้อ: 1. แม่เก็บลูกเป็น private | 2. แม่ new ลูกเอง | 3. คนอื่นห้ามแตะลูกตรงๆ | 4. แม่ตาย ลูกตายด้วย (items.clear())

หน้า 3 (สไลด์ 329): ตัวอย่างของระบบเรา (src/model/Order.java)

สไลด์ที่ 3 | สไลด์ 329 (Canva)

⏱️ เวลาพูด: ~45 วินาที • 🎯 วัตถุประสงค์: ชี้ให้เห็นโค้ดจริงว่าบรรทัดไหนคือ Composition บรรทัดไหนคือ Association

---

👁️ **ท่าทางและการชี้จอ:** 1. ชี้บรรทัดที่ 1: `private ArrayList<OrderItem> items;`
2. ชี้บรรทัดที่ 2: `private Table table;`
3. ชี้ในเมธอด `destroy()` ตรง `item.destroy()` และ `table = null;`

"สไลด์หน้านี้คือโค้ดจริงในโปรเจกต์ของเราจากคลาส `Order.java` เพื่อให้เห็นการนำทฤษฎีไปแปลงเป็นโค้ดอย่างชัดเจนครับ:"

"สังเกตที่บรรทัดบนสุด: `Order` ถือ `items` ซึ่งเป็น `OrderItem` เป็น **Composition** สังเกตว่าในเมธอด `addItem()` ตัวแม่จะเป็นคน `new OrderItem` ขึ้นมาเอง และเมื่อเกิดการยกเลิกในเมธอด `destroy()` ตัว Order จะวนลูปสั่ง `item.destroy()` และสั่ง `items.clear()` ทำให้ OrderItem ทั้งหมดถูกทำลายไปพร้อมกัน"

"แต่ในขณะเดียวกัน `Order` ก็ถือ `Table table` ซึ่งเป็นเพียง **Association (การอ้างถึง)** เมื่อออเดอร์ถูกลบ เราแค่เซต `table = null` เพื่อตัดการเชื่อมโยง แต่ตัว Object โต๊ะอาหารยังคงอยู่ในระบบ ไม่ได้ถูกทำลายตาม และพร้อมรับลูกค้ารายถัดไปได้ทันทีครับ"

เน้นย้ำ: ชี้ให้เห็นความแตกต่างชัดเจนระหว่าง `items.clear()` (Composition) กับ `table = null` (Association)

ACT 2: สาธิตการทำงาน Text Mode Simulator (FR1 – FR8)

หน้า 4 – 13 | รวมเวลา ~3.0 - 3.5 นาที

หน้า 4: คั่นหมวด Text Mode Simulator

⏱️ เวลาพูด: ~10 วินาที • 🎯 วัตถุประสงค์: เกริ่นนำเข้าสู่การรันโปรแกรมจริง

สไลด์ที่ 4 | PDF หน้า 4

"ต่อไปจะเข้าสู่การสาธิตการรันจริงผ่าน **Text Mode Simulator** ซึ่งจำลองการทำงานทั้ง 8 Use Cases ตั้งแต่สั่งอาหาร ปรุง เสิร์ฟ จ่ายเงิน ไปจนถึงออกรายงานยอดขายครับ"

หน้า 5 (สไลด์ 331): FR1 (1.1 สแกน QR ประจำโต๊ะ & 1.2 เลือกเมนู ปรับแต่งระดับเผ็ด)

⏱️ เวลาพูด: ~35 วินาที • 🎯 วัตถุประสงค์: แสดงการเริ่มออเดอร์ ผูกสมาชิก และตัวเลือกเสริม

สไลด์ที่ 5 | สไลด์ 331 (Canva)

👁️ **ท่าทางและการชี้จอ:** ชี้กล่องซ้ายตรงการผูกเบอร์สมาชิก MBR-0001 แล้วชี้กล่องขวาตรงยอด 168.00 บาท

"เริ่มต้นที่ **FR1 การสั่งอาหารผ่าน QR Code:**"

"**ฝั่งซ้าย (1.1):** จำลองลูกค้าสแกน QR ประจำโต๊ะ 4 กรอกเบอร์โทรศัพท์ 099-999-9999 ระบบจะทำการค้นหาและผูกกับสมาชิกชื่อคุณสมชาย (MBR-0001) และสร้างตะกร้าอาหาร `CRT-20261101-0001` ให้อัตโนมัติ"

"**ฝั่งขวา (1.2):** ลูกค้าเข้าสู่ขั้นตอนการเลือกเมนู เช่น เลือกหมวดผัด เมนูผัดกะเพราปลา 2 จาน สามารถระบุระดับความเผ็ด 'เผ็ดมาก' และเพิ่มท็อปปิ้ง 'ไข่ดาว +15 บาท' ระบบจะคำนวณราคารวมเป็น 168 บาทอย่างแม่นยำครับ"

หน้า 6 (สไลด์ 332): FR1 (1.3 ตรวจสต็อกตาม Recipe & 1.4 ติดตามสถานะออเดอร์)

สไลด์ที่ 6 | สไลด์ 332 (Canva)

⏱️ เวลาพูด: ~40 วินาที • 🎯 วัตถุประสงค์: โชว์ Two-Pass Stock Check และระบบแยกสถานีครัว

👁️ **ท่าทางและการชี้จอ:** ชี้ที่กล่องซ้ายตรง 'ตรวจสอบสต็อกตามสูตร Recipe' แล้วชี้กล่องขวาตรง 'รอปรุง (ครัวร้อน)'

"**ฝั่งซ้าย (1.3):** เมื่อลูกค้ากดยืนยันออเดอร์ยอดรวม 233 บาท ระบบจะใช้กลไกสำคัญคือ **Two-Pass Stock Check** ในคลาส `Order` โดยจะวนลูปตรวจสอบวัตถุดิบทุกชนิดตามสูตร `Recipe` (เนื้อปลา, ไข่ไก่, หมูยอ) ให้มั่นใจว่าสต็อกมีเพียงพอครบ 100% ก่อน เมื่อผ่านทั้งหมดจึงจะยอมตัดสต็อกจริง ป้องกันปัญหาหักสต็อกไปครึ่งหนึ่งแล้วของหมดครับ"

"**ฝั่งขวา (1.4):** หน้าจอติดตามสถานะ แสดงให้เห็นว่าออเดอร์ถูกแยกรายการอาหารส่งไปยังสถานีครัวร้อนและครัวเย็นโดยอัตโนมัติ พร้อมแสดงเวลารวมตาม SOP คือ 8.0 นาทีครับ"

หน้า 7 (สไลด์ 333): FR2 (2.1 รายการคู่มือ SOP & 2.2 กำหนดขั้นตอนย่อย SOPStep)

สไลด์ที่ 7 | สไลด์ 333 (Canva)

⏱️ เวลาพูด: ~30 วินาที • 🎯 วัตถุประสงค์: แสดงการควบคุมมาตรฐานการทำงาน SOP

👁️ **ท่าทางและการชี้จอ:** ชี้ที่เวลารวมมาตรฐาน 2.0 นาที และปุ่มกดเผยแพร่ (Publish)

"ต่อมาที่ **FR2 การจัดการคู่มือมาตรฐาน SOP:**"

"ผู้จัดการสามารถตรวจสอบและจัดการคู่มือการปรุงอาหารของร้านได้ โดยในฝั่งขวา (2.2) สามารถกำหนดขั้นตอนย่อย `SOPStep` เช่น เมนูชาใหม่ มีขั้นตอนต้มน้ำร้อน 1.5 นาที ผสมนม 0.5 นาที รวมเป็น 2.0 นาที เมื่อขั้นตอนสมบูรณ์แล้ว ผู้จัดการจึงกด 'เผยแพร่' (Publish) เพื่อส่งต่อให้ครัวนำไปปฏิบัติครับ"

หน้า 8 (สไลด์ 334): FR3 (3.1 จอครัว KDS ดู SOP & 3.2 บันทึกเวลาจริงเทียบ SOP)

สไลด์ที่ 8 | สไลด์ 334 (Canva)

⏱️ เวลาพูด: ~35 วินาที • 🎯 วัตถุประสงค์: แสดงการจับเวลาจริง เทียบเป้าหมาย SOP และการเสิร์ฟ

👁️ **ท่าทางและการชี้จอ:** ชี้ที่ 'เวลาที่ใช้จริง: 6.0 นาที (เป้าหมาย 7.0 นาที) [ตรงเวลา]'

"**FR3 ระบบจอครัว KDS และการเสิร์ฟ:**"

"**ฝั่งซ้าย (3.1):** คนครัวเปิดดูคิวอาหาร ดูขั้นตอนและเวลามาตรฐาน 7.0 นาที จากนั้นกด 'เริ่มลงมือปรุง' เพื่อให้ระบบเริ่มจับเวลา"

"**ฝั่งขวา (3.2):** เมื่อปรุงเสร็จ ระบบบันทึกเวลาจริงได้ 6.0 นาที ซึ่งเร็วกว่าเวลามาตรฐาน SOP ระบบเปลี่ยนสถานะเป็น 'พร้อมเสิร์ฟ' เพื่อให้พนักงานนำไปเสิร์ฟที่โต๊ะ และส่งออเดอร์ไปยังแคชเชียร์เพื่อรอชำระเงินครับ"

หน้า 9 (สไลด์ 335): FR4 (4.1 ตรวจสอบบิล แลกแต้ม & 4.2 พิมพ์ใบเสร็จและสะสมแต้ม)

สไลด์ที่ 9 | สไลด์ 335 (Canva)

⏱️ เวลาพูด: ~35 วินาที • 🎯 วัตถุประสงค์: แสดงการชำระเงิน คำนวณเงินทอน และสะสมแต้ม

👁️ **ท่าทางและการชี้จอ:** ชี้ที่ยอดสุทธิ 70 บาท, เงินทอน 30 บาท และแต้มสะสมเพิ่ม 1 แต้ม

"**FR4 การชำระเงินและระบบแต้มสะสม:**"

"แคชเชียร์เรียกบิลโต๊ะ 1 ยอดรวม 80 บาท ลูกค้าขอแลกแต้มสะสม 10 แต้ม ได้ส่วนลด 10 บาท ยอดสุทธิจึงเหลือ 70 บาท ลูกค้าจ่ายด้วยเงินสด 100 บาท ระบบคำนวณเงินทอน 30 บาท พร้อมออกใบเสร็จ `RCT-PAY-20261101-0001` และทำการคำนวณสะสมแต้มใหม่เพิ่ม 1 แต้มเข้าบัญชีสมาชิกทันทีครับ"

หน้า 10 (สไลด์ 336): FR5 (5.1 ตรวจสอบสต็อก ออกใบ PO & 5.2 ตรวจรับเข้าคลังและ Audit Log)

สไลด์ที่ 10 | สไลด์ 336 (Canva)

⏱️ เวลาพูด: ~35 วินาที • 🎯 วัตถุประสงค์: โชว์การออกใบสั่งซื้อ และการบันทึกประวัติสต็อก

> 👁️ **ท่าทางและการชี้จอ:** ชี้ที่การแจ้งเตือนสต็อกต่ำกว่าเกณฑ์ และประวัติ StockMovement (RECEIVE, ADJUST)

> "**FR5 การจัดการสต็อกและใบสั่งซื้อ PO:**"
>
> "ระบบตรวจพบเนื้อปลาเหลือ 1.3 กก. ต่ำกว่าเกณฑ์สั่งซื้อ 2.0 กก. ระบบจึงแจ้งเตือน ผู้จัดการจึงออกใบสั่งซื้อ `PO-21` จำนวน 2.0 กก. เมื่อของมาส่ง ระบบตรวจรับสินค้า ยอดสต็อกเพิ่มเป็น 7.3 กก. และบันทึกประวัติ `StockMovement` (RECEIVE) และเมื่อผู้จัดการตรวจนับจริงพบของเสีย จึงปรับยอดเหลือ 6.0 กก. โดยบันทึกประวัติแบบ ADJUST เป็น Audit Log ย้อนหลังได้สมบูรณ์ครับ"

หน้า 11 (สไลด์ 337 / 339): FR6 (6.1 สรุปยอดขายรวม & 6.2 รายงานอันดับขายดีและสถิติ SOP)

สไลด์ที่ 11 | สไลด์ 337 (Canva)

⏱️ เวลาพูด: ~30 วินาที • 🎯 วัตถุประสงค์: แสดงรายงานเชิงลึกสำหรับผู้บริหาร

> 👁️ **ท่าทางและการชี้จอ:** ชี้ที่การแยกช่องทางเงินสด 70 บาท และสถิติเสิร์ฟตรงเวลา 100%

> "**FR6 รายงานและแผงควบคุมภาพรวมร้าน:**"
>
> "ผู้จัดการสามารถดูสรุปยอดขายประจำวันแบบเรียลไทม์ แยกช่องทางเงินสดและ QR และดูอันดับอาหารขายดี พร้อมสถิติประสิทธิภาพการปรุงอาหารตาม SOP ซึ่งพบว่าเสิร์ฟตรงเวลา 100% และความเร็วการปรุงเร็วกว่ามาตรฐาน 1.0 นาที ประเมินผลระดับดีเยี่ยมครับ"

หน้า 12 (สไลด์ 338): FR7 & FR8 (7.1 จัดการเมนูและสูตร & 8.1 ระบบสมาชิกและกฎแต้ม)

สไลด์ที่ 12 | สไลด์ 338 (Canva)

⏱️ เวลาพูด: ~35 วินาที • 🎯 วัตถุประสงค์: แสดงการควบคุมเวอร์ชันสูตร และกฎแต้มกลางร้าน

👁️ **ท่าทางและการชี้จอ:** ชี้ที่ Recipe Version 2 และเมธอด setRule() กำหนดเงื่อนไขแต้มใหม่

"และสองข้อกำหนดสุดท้าย **FR7 และ FR8:**"

"**ฝั่งซ้าย (7.1):** การจัดการเมนูและสูตร รองรับการสร้างสูตรอาหารหลายเวอร์ชัน เช่น Recipe Version 2 โดยออเดอร์เก่าที่เคยสั่งไปแล้วจะยังคงผูกกับ Version 1 เดิม ป้องกันปัญหาข้อมูลในอดีตคลาดเคลื่อน"

"**ฝั่งขวา (8.1):** ระบบสมาชิก ผู้จัดการสามารถเปลี่ยนกฎการคำนวณแต้มกลางร้านผ่านเมธอด `setRule()` เช่น เปลี่ยนเป็นทุก 100 บาทได้ 1 แต้ม มีผลกับการชำระเงินครั้งถัดไปทันทีครับ"

ACT 3: การพิสูจน์ Composition 5 คู่แท้เชิงประจักษ์ หน้า 14 – 19 | รวมเวลา ~2.5 - 3 นาที

หน้า 14: คั่นหมวดตัวอย่าง Composition 5 คู่แท้

สไลด์ที่ 14 | PDF หน้า 14

⏱️ เวลาพูด: ~10 วินาที • 🎯 วัตถุประสงค์: ดึงความสนใจเข้าสู่การพิสูจน์เชิงลึก

"ในส่วนสุดท้ายนี้ เราจะมาพิสูจน์ความสัมพันธ์แบบ **Composition ครบทั้ง 5 คู่แท้** ที่เกิดขึ้นจริงในระบบ เพื่อยืนยันว่าการออกแบบคลาสของเราสอดคล้องตามหลักการ OOP 100% ครับ"

หน้า 15 (สไลด์ 340): Composition ① Order ◆ OrderItem

คู่ที่ 1 | สไลด์ 340 (Canva)

⏱️ เวลาพูด: ~40 วินาที • 🎯 วัตถุประสงค์: พิสูจน์ว่า OrderItem ถูกลบหายตาม Order

👁️ **ท่าทางและการชี้จอ:** ชี้ที่กล่อง Terminal ตรง '2 -> 0 รายการ' และชี้โค้ด `items.clear()`

"**คู่ที่ 1: `Order` เป็นเจ้าของ `OrderItem` **"

"เริ่มต้นออเดอร์ `ORD-001` ถือรายการอาหาร `OrderItem` อยู่จำนวน 2 รายการ"

"เมื่อมีการเรียกคำสั่งยกเลิกออเดอร์ผ่าน `order.destroy()` ในโค้ดจริง เมธอดจะทำการวนลูปสั่ง `item.destroy()` ของลูกทุกตัว และสั่ง `items.clear()` "

"**ผลลัพธ์เชิงประจักษ์คือ:** รายการอาหารในออเดอร์ลดลงจาก 2 รายการเหลือ 0 รายการทันที พิสูจน์ว่าส่วนประกอบย่อยตายตามเจ้าของ ในขณะที่ Table และ Customer ยังคงอยู่ในระบบ เพราะเป็นเพียง Association ครับ"

หน้า 16 (สไลด์ 341): Composition ② Menu ◆ Recipe

คู่ที่ 2 | สไลด์ 341 (Canva)

⏱️ เวลาพูด: ~30 วินาที • 🎯 วัตถุประสงค์: พิสูจน์ว่าสูตรอาหารตายตามเมนู

👁️ **ท่าทางและการชี้จอ:** ชี้ที่ Recipe 2 เวอร์ชัน ลดเหลือ 0 สูตร และเน้นว่า Ingredient ยังอยู่

"**คู่ที่ 2: `Menu` เป็นเจ้าของ `Recipe` (สูตรอาหาร)**"

"เมนูข้าวต้มปลาถือสูตรอาหารอยู่ 2 เวอร์ชัน เมื่อผู้จัดการสั่งลบเมนูผ่าน `menu.destroy()` เมธอดจะสั่ง `recipes.clear()` ทำให้สูตรอาหารทั้งสองเวอร์ชันถูกทำลายตามเมนูไปทันที เหลือ 0 สูตร ส่วนวัตถุดิบเนื้อปลาในคลังยังคงอยู่ตามปกติ เพราะ Recipe แค่อ้างถึงวัตถุดิบแบบ Association ครับ"

หน้า 17 (สไลด์ 342): Composition ③ SOP ◆ SOPStep

คู่ที่ 3 | สไลด์ 342 (Canva)

⏱️ เวลาพูด: ~30 วินาที • 🎯 วัตถุประสงค์: พิสูจน์ว่าขั้นตอนย่อยตายตามคู่มือ SOP

👁️ **ท่าทางและการชี้จอ:** ชี้ที่ขั้นตอนย่อย 2 ขั้นตอน ลดเหลือ 0 ขั้นตอน

"**คู่ที่ 3: `SOP` เป็นเจ้าของ `SOPStep` (ขั้นตอนย่อยมาตรฐาน)**"

"คู่มือ SOP ข้าวต้มปลามีขั้นตอนย่อย 2 ขั้นตอน เมื่อคู่มือถูกทำลายใน `sop.destroy()` จะมีการสั่ง `steps.clear()` ทำให้ขั้นตอนย่อยทั้งสองขั้นตอนหายไปพร้อมคู่มือทันที เหลือ 0 ขั้นตอนครับ"

หน้า 18 (สไลด์ 343): Composition ④ Ingredient ◆ StockMovement

คู่ที่ 4 | สไลด์ 343 (Canva)

⏱️ เวลาพูด: ~30 วินาที • 🎯 วัตถุประสงค์: พิสูจน์ว่าประวัติสต็อกเฉพาะตายตามวัตถุดิบ

👁️ **ท่าทางและการชี้จอ:** ชี้ประวัติสต็อก 3 รายการ ลดเหลือ 0 บันทึก

"**คู่ที่ 4: `Ingredient` เป็นเจ้าของ `StockMovement` (ประวัติความเคลื่อนไหวสต็อก)**"

"วัตถุดิบเนื้อปลามีประวัติความเคลื่อนไหวสต็อก 3 รายการ เมื่อวัตถุดิบถูกลบออกจากระบบ `ingredient.destroy()` จะทำการล้าง `movements.clear()` ทำให้ประวัติการเคลื่อนไหวเฉพาะของวัตถุดิบนั้นถูกทำลายตามไปด้วย เหลือ 0 รายการครับ"

หน้า 19 (สไลด์ 344): Composition ⑤ Member ◆ PointTransaction

คู่ที่ 5 | สไลด์ 344 (Canva)

⏱️ เวลาพูด: ~40 วินาที • 🎯 วัตถุประสงค์: พิสูจน์คู่สุดท้าย และกล่าวสรุปจบอย่างน่าประทับใจ

> 👁️ **ท่าทางและการชี้จอ:** ชี้ประวัติแต้ม 3 รายการ เหลือ 0 บันทึก แล้วสบตาอาจารย์เพื่อกล่าวปิด

"และ **คู่ที่ 5: `Member` เป็นเจ้าของ `PointTransaction` (ประวัติแต้มสะสม)**"

"สมาชิกคุณสมชายมีประวัติแต้ม 3 รายการ เมื่อมีการลบบัญชีสมาชิก `member.destroy()` จะทำการสั่ง `transactions.clear()` และรีเซตแต้มเป็น 0 ทำให้ประวัติแต้มของผู้ใช้รายนั้นหายไปพร้อมกับบัญชีสมาชิกทันทีครับ"

"การพิสูจน์ทั้ง 5 คู่นี้ แสดงให้เห็นว่าโครงสร้างโค้ด Java ของเราตอบโจทย์ทั้งการใช้งานจริงตามข้อกำหนด และเป็นไปตามหลักการออกแบบ Object-Oriented Analysis and Design อย่างสมบูรณ์แบบครับ"

"กลุ่มของพวกเราขอจบการนำเสนอ Phase 9 เพียงเท่านี้ อาจารย์และเพื่อนๆ มีข้อเสนอแนะหรือคำถามเพิ่มเติมไหมครับ ขอบคุณครับ"

🛡️ คลังคำถาม-คำตอบปราบเซียน (Q&A Defense Guide)

เตรียมพร้อมรับมือทุกคำถามจากอาจารย์

❓ คำถาม 1: ใน Java ไม่มีคำสั่ง delete เหมือนภาษา C++ แล้วเราทำ Composition อย่างไร?

แนวทางการตอบ: "ในภาษา Java การจัดการหน่วยความจำอยู่ภายใต้การดูแลของ **Garbage Collector** ครับ เราจึงพิสูจน์ความเป็น Composition ผ่าน **Life-cycle Method** นั่นคือเมธอด `destroy()` โดยเมื่อ Object ตัวแม่ถูกทำลาย ตัวแม่จะเป็นคนสั่ง `clear()` และตัดการอ้างอิง Object ลูกทั้งหมด ส่งผลให้ Object ลูกกลายเป็น Unreachable Object และถูก Garbage Collector เก็บกวาดทำลายหน่วยความจำตามไปทันทีครับ"

❓ คำถาม 2: Two-Pass Stock Check ใน FR1 คืออะไร ทำไมต้องทำสองรอบ?

แนวทางการตอบ: "Two-Pass Check คือการแยกกระบวนการตรวจสอบและตัดสต็อกออกเป็น 2 ลูปครับ: **รอบที่ 1 (Validation Pass):** วนลูปตรวจสอบวัตถุดิบทุกชนิดที่ต้องใช้ในออเดอร์นั้นให้ครบ 100% ก่อนว่าพอหรือไม่ และ **รอบที่ 2 (Deduction Pass):** เมื่อผ่านรอบแรกครบทั้งหมดแล้ว จึงจะทำการตัดสต็อกจริง เพื่อป้องกันปัญหาตัดสต็อกชิ้นแรกไปแล้ว แต่ของชิ้นถัดไปหมด ทำให้ระบบค้างและสต็อกเสียหายครับ"

❓ คำถาม 3: ทำไม OrderItem ต้องเก็บสำเนาของราคาและสูตรเป็นของตัวเอง ทั้งที่มี Menu อยู่แล้ว?

แนวทางการตอบ: "เพื่อป้องกันปัญหา **Historical Data Drift** ครับ ในชีวิตจริง ถ้าร้านค้าปรับราคาเมนูผัดกะเพราจาก 69 บาทเป็น 79 บาทในวันพรุ่งนี้ บิลย้อนหลังที่ลูกค้าเคยสั่งไปเมื่อวานจะต้องแสดงราคา 69 บาทเท่าเดิม ไม่ควรเปลี่ยนตาม ดังนั้น `OrderItem` จึงต้องทำ Snapshot ราคาและสูตร ณ เวลาที่สั่งเก็บไว้ในตัวเองครับ"

❓ คำถาม 4: ทำไมความสัมพันธ์ระหว่าง Order กับ Table หรือ Customer ถึงเป็นแค่ Association ไม่ใช่ Composition?

แนวทางการตอบ: "เพราะเรื่อง **Lifecycle และ Ownership** ครับ โต๊ะอาหาร (Table) และ ลูกค้า (Customer) มีตัวตนอยู่จริงในร้านค้ามาก่อนที่ออเดอร์จะเกิด และเมื่อออเดอร์นั้นถูกชำระเงินหรือยกเลิกทิ้งไป โต๊ะก็ยังคงตั้งอยู่ที่ร้าน และลูกค้าก็ยังคงเป็นสมาชิกของร้าน ไม่ได้พังหรือหายไปตามออเดอร์ ดังนั้นความสัมพันธ์จึงเป็นแค่ Association (การอ้างถึง) ครับ"

❓ คำถาม 5: ทำไมโปรเจกต์นี้ถึงไม่มีการใช้ Framework เช่น Spring Boot หรือ Lombok?

แนวทางการตอบ: "เพราะเป้าหมายของวิชา OOAD คือการศึกษาและพิสูจน์ความเข้าใจในกลไกพื้นฐานของ **Object-Oriented Programming** ครับ การใช้ Framework หรือ Annotation สำเร็จรูปจะบดบังกลไกของ Constructor, Encapsulation, Reference, และ Garbage Collection การเขียนด้วย Core Java ล้วนๆ ทำให้เราสามารถอธิบายการทำงานได้ครบทุกบรรทัดอย่างโปร่งใสครับ"